数学与 CS 问答笔记

answer-note

今天讲思考表示不变式（RI）的六个层面：值、结构、生命周期、边界、规格、工程；每个层面给核心问题、典型子句、Go 检查方式与失败后果，并对 `mem` 只列问题、不填答案。

October 26, 2026

Contents

1. 思考 RI 的六个层面（Six Layers for Reasoning About Rep Invariants） 4
   1.1. 费曼直觉 4
   1.2. 硬核机制 4
      1.2.1. 层面一：值层（field-level） 4
      1.2.2. 层面二：结构层（cross-field / container-level） 4
      1.2.3. 层面三：生命周期层（state machine） 5
      1.2.4. 层面四：边界层（rep exposure） 5
      1.2.5. 层面五：规格层（RI 与 AF/规格的关系） 5
      1.2.6. 层面六：工程层（checkability） 6
   1.3. 六个层面汇总 6
   1.4. 对 `mem` 的六个层面（只列问题，不填答案） 6
   1.5. 陷阱与易错 7
   1.6. 关键点 7
   1.7. 变式检验 7
   1.8. 同类归类 7
   1.9. Go 语言机制补充 8
   1.10. 本次操作与当前状态 8
   1.11. 单键谓词与键集合的全称量化 8
      1.11.1. 一、两种视角其实是“谓词”与“量词” 8
      1.11.2. 二、三层分离 8
      1.11.3. 三、带背景的例子：用户索引表 9
      1.11.4. 四、前置条件与 RI：同一个谓词，两个使用点 10
      1.11.5. 五、写 RI 时的分类法 10
      1.11.6. 六、对 `mem` 的映射（只列问题） 11
      1.11.7. 本次操作 11
      1.11.8. 谓词、量化与三层含义 11
2. 讲解偏好：例子要带背景 15
   2.1. 示范一：`RatNum` 的 RI（结构层） 15
   2.2. 示范二：map 的空 key（值层） 15
   2.3. 本次操作 16
3. 测试分区的顺序：先 subdomain，后 cartesian product 17
   3.1. 费曼直觉 17
   3.2. 硬核机制 17
      3.2.1. 一、三个概念 17
      3.2.2. 二、推荐顺序 17
      3.2.3. 三、为什么先 subdomain 17
      3.2.4. 四、什么时候必须做笛卡尔积 18
      3.2.5. 五、裁剪策略 18
      3.2.6. 六、带背景的例子：转账函数 18
      3.2.7. 七、与 RI 的类比 19
      3.2.8. 八、对当前项目的问题清单（不填答案） 19

3.3. Go 语言机制补充 20
3.4. 本次操作与当前状态 20
3.5. 边界属于哪个阶段：分区，而不是列维度 20
3.5.1. 一、为什么不是“列维度” 20
3.5.2. 二、材料里的做法 20
3.5.3. 三、操作顺序 21
3.5.4. 四、带背景的例子：分页函数 21
3.5.5. 五、对当前项目的边界问题清单（不填答案） 21
3.5.6. 六、常见错误 21
3.5.7. 七、与 RI 的类比 22
3.6. 本次操作与当前状态 22
3.7. Using multiple partitions 与 cartesian product 的关系 22
3.7.1. 一、材料怎么说 22
3.7.2. 二、两者的分工 23
3.7.3. 三、它们各自属于哪个阶段 23
3.7.4. 四、带背景的例子：乘法 23
3.7.5. 五、操作建议 24
3.7.6. 六、常见错误 24
3.8. 本次操作与当前状态 24
3.9. 操作分类（creator / producer / observer / mutator）在哪个阶段 25
3.9.1. 一、四类操作 25
3.9.2. 二、放到五步框架里 25
3.9.3. 三、为什么不能把操作类型当维度 26
3.9.4. 四、带背景的例子：Counter ADT 26
3.9.5. 五、与 RI 的对应 27
3.9.6. 六、常见错误 27
3.10. 本次操作与当前状态 27
3.11. Testing an abstract data type 与五步的区别和联系 27
3.11.1. 一、材料怎么说 27
3.11.2. 二、区别 28
3.11.3. 三、联系：ADT 测试多出的四件事 29
3.11.4. 四、带背景的例子：MyString 29
3.11.5. 五、常见错误 30
3.12. 本次操作与当前状态 30
Index of Listings 31

1. 思考 RI 的六个层面（Six Layers for Reasoning About Rep Invariants）

1.1. 费曼直觉

写 RI 最怕“想到哪写到哪”。更稳的做法是从内到外过六道关，像体检一样：

1. **值层**：每个单项指标正常吗?
2. **结构层**：指标之间的关系正常吗?
3. **生命周期层**：不同时间点的状态都正常吗?
4. **边界层**：有没有外部因素能污染指标?
5. **规格层**：这套标准与“什么算健康”的定义一致吗？
6. **工程层**：这套检查真的能执行吗?

六层合起来，才是一份既能挡 bug、又能被代码检查、还不会误伤合法输入的 RI。

1.2. 硬核机制

1.2.1. 层面一：值层（field-level）

核心问题：单独看每一个字段，哪些取值是非法的?

典型子句：

1. 非 nil（map、slice、指针）；
2. 数值范围（`n >= 0`、`den > 0`）；
3. 格式/长度（字符串非空、长度上限）；
4. 枚举合法值（状态字段只能是几个常量之一）。

```
// Rep invariant:
//   n >= 0
```

Listing 1: 值层的最小例子：单个字段的范围约束；`Counter` 是通用示例

Go 检查：直接比较，成本 $O(1)$。**失败后果**：单个字段非法，AF 在这个 rep 值上没有定义。

1.2.2. 层面二：结构层（cross-field / container-level）

核心问题：多个字段之间、或容器内部的元素之间，必须满足什么关系?

典型子句：

1. 跨字段：`len(keys) == len(vals)`、`min <= max`、`num/den` 已约分；
2. 容器内部：slice 有序、元素非 nil、无重复（如果容器本身不保证）；
3. 嵌套结构：map 的值如果是 slice，元素是否允许 nil。

```
// Rep invariant:
//   den > 0
//   gcd(|num|, den) = 1
```

Listing 2: 结构层的例子：第二条约束两个字段之间的关系；`RatNum` 来自课程材料

Go 检查：遍历容器，成本 $O(n)$。**失败后果**：结构不一致时，AF 可能给出错误的抽象值（而不是无定义）。

> ⚠️ **陷阱预警** map 的键唯一是**语言保证**，不需要写进 RI；但“键的格式/长度/范围”仍可能需要写。不要把语言已经保证的东西重复进 RI，那只会增加噪音。

1.2.3. 层面三：生命周期层（state machine）

核心问题：对象在创建前、创建后、操作中、关闭后，合法状态的集合是否不同？

典型子句：

1. 创建前：零值是否可用？`nil` 字段是否合法？
2. 创建后：哪些字段必须已初始化？
3. 关闭后：对象是否还能调用方法？
4. 方法内部：允许暂时破坏 RI，但返回前必须恢复。

```
// Rep invariant:
//   opened == true after Open
//   items != nil after Open
```

Listing 3: 生命周期层的示意：同一个 rep 在不同阶段有不同的合法状态；字段名是占位，不是项目代码

Go 检查：状态字段或 nil 判断，成本 $O(1)$。**失败后果**：同一方法在不同阶段行为不一致，调用方无法推理。

> 📌 **核心要点** 生命周期层与“保持”义务直接相关：creator 建立初始合法状态，mutator 在返回前恢复合法状态；方法内部可以短暂不合法（例如先删后补），但**不能跨越返回点**。

1.2.4. 层面四：边界层（rep exposure）

核心问题：客户端能不能绕过方法，拿到并修改 rep 的可变部分？

典型情形：

1. 返回内部 slice/map；
2. 保存调用方传入的 slice/map；
3. 通过接口或指针间接共享内部对象。

这一层严格说属于**安全性论证**，但它与 RI 强联动：一旦 rep 被客户端改坏，RI 的“保持”义务就失效了——因为改变不是通过你的方法发生的。

Go 检查：代码审查 + 防御性拷贝（`bytes.Clone`、`slices.Clone`、`maps.Clone`）或文档约定。**失败后果**：RI 在方法之外被破坏，`checkRep` 只能事后发现，找不到来源。

1.2.5. 层面五：规格层（RI 与 AF/规格的关系）

核心问题：这条 RI 排除的 rep 值，是否对应规格允许的抽象值？

三条关系：

1. RI 是 AF 的定义域：RI 为真 $\iff$ AF 有定义；
2. RI 过强：排除了规格允许的抽象值，AF 不再满射；
3. RI 过弱：非法 rep 溜进来，AF 在它上面没有定义。

RI 也不能偷偷替代前置条件：前置条件是“调用方必须保证什么”，RI 是“实现内部必须始终成立什么”。把前者写进后者，会改变规格。

检查方式：把 RI 排除的 rep 值逐个映射回抽象值，问“规格允许它吗？”。**失败后果**：合法抽象值造不出来，或非法状态进入。

1.2.6. 层面六：工程层（checkability）

核心问题：这条 RI 真的能被检查吗？成本可接受吗？换表示后还成立吗？

四个子问题：

1. **可写**：每条子句能否落成 `checkRep` 里的一个 `if`？
2. **成本**：$O(1)$ 还是 $O(n)$？每次 mutator 都调用是否可接受？
3. **可测**：能否在测试里“随机操作 + 调用 checkRep”形成回归网？
4. **可维护**：这条 RI 是否绑定了当前表示？换 rep 时它是否必须重写？

失败后果：注释与代码不一致；或者检查太贵，实际从不调用，等于没有。

1.3. 六个层面汇总

层面	核心问题	典型子句	检查成本	失败后果
值层	单个字段哪些值非法？	`den > 0`、`s != nil`	$O(1)$	AF 无定义
结构层	字段/元素之间必须满足什么关系？	有序、无重复、`gcd = 1`	$O(n)$	AF 给出错误值
生命周期层	不同阶段合法状态是否不同？	创建后才能用、关闭后不可用	$O(1)$	行为随阶段漂移
边界层	客户端能否绕过方法改 rep?	不返回内部 slice、保存时拷贝	审查/拷贝	RI 在方法外被破坏
规格层	RI 是否与规格一致？	不过强、不过弱	推理/对照	合法值造不出或非法值溜入
工程层	能检查、能测试、能维护吗？	每条对应一个 `if`、成本可控	实现/测试	写了不查或查不起

1.4. 对 `mem` 的六个层面（只列问题，不填答案）

1. **值层**：`mem` 本身为 nil 算合法吗？value 为 nil 算合法吗？
2. **结构层**：map 的键唯一已由语言保证；除此之外还需要什么关系（键的格式、值的长度）？

3. **生命周期层**：`Open` 之前与之后，合法状态的集合是否不同？`Close` 之后呢？
4. **边界层**：`Get` 返回的切片与 `mem` 共享底层数组吗？`Set` 保存调用方切片吗？
5. **规格层**：空键、空值、nil 值在抽象值空间里有意义吗？
6. **工程层**：`checkRep` 遍历整个 map 的成本可接受吗？在哪些方法末尾调用？

> ⚠️ **陷阱预警** 这六行是**问题清单**，不是答案。每一行都可能导出零条或多条 RI 子句；某一行没有约束，也是合法的设计结论。不要为了“六层都填满”而硬造规则。

1.5. 陷阱与易错

1. **只做值层**：忘了结构关系与生命周期，RI 看起来完整，实际漏掉大半非法状态；
2. **把边界层写进 RI**：“值不共享”是安全性论证，不是 rep 的合法性条件；
3. **把规格当前置条件**：RI 不是“调用方必须传什么”，而是“内部必须始终成立什么”；
4. **过强**：排除规格允许的抽象值，AF 不满射；
5. **过弱**：非法 rep 溜进来，AF 无定义；
6. **重复语言保证**：map 键唯一、slice 长度非负，这些不需要写；
7. **写了不检查**：注释三条、`checkRep` 一条；
8. **检查太贵**：$O(n)$ 的检查放在每次观察者调用里，最后被注释掉。

1.6. 关键点

一句话：**六个层面是从内到外的过滤网——值层管单项，结构层管关系，生命周期层管阶段，边界层管旁路，规格层管一致性，工程层管可执行；缺哪层，RI 就在哪层漏。**

1.7. 变式检验

1. 如果 RI 只覆盖值层，会漏掉哪些类型的非法状态？举两个通用例子。
2. 生命周期层允许“方法内部暂时破坏 RI”吗？这与“保持”义务矛盾吗？
3. 边界层严格说属于安全性论证，为什么仍然要放进思考 RI 的框架里？
4. 一条 $O(n)$ 的 RI 检查，每次 mutator 都调用可能太贵；有哪些工程上的替代方案？
5. 语言已经保证 map 键唯一，为什么值层/结构层仍可能需要检查键的格式？

1.8. 同类归类

层面	课程位置	Go 机制	技能
值层	6.102 R07 RI	nil、零值、范围	`golang-safety`
结构层	6.102 R07 RI	slice/map 内部关系	`golang-data-structures`
生命周期层	6.031 R11 建立/保持	`Open` / `Close` 、零值	`golang-safety`
边界层	6.102 R07 rep exposure	切片别名、`Clone`	`golang-safety`
规格层	6.102 R04/R05	前置/后置条件	`golang-error-handling`
工程层	6.102 R07 `checkRep`	`panic` 、测试	`golang-testing`

1.9. Go 语言机制补充

1. map 的零值是 nil：读合法、写 panic（`/usr/lib/go/src/runtime/map.go:21`）；“nil 是否合法”通常属于生命周期层或值层。
2. map 的键必须可比较，且键唯一由语言保证；`[]byte` 不能直接做键。
3. 切片是描述符；共享底层数组时一端修改会影响另一端（`go doc slices.Clone`、`bytes.Clone`）。
4. Go 没有 `assert`；`checkRep` 用 `if !cond { panic(...) }`，测试用 `t.Fatalf`。
5. 同包测试可以直接调用 `checkRep`，也可以直接改字段——后者绕过 RI，属于白盒滥用。
6. `go doc ./internal/db` 只显示导出文档；字段旁注释不会出现在里面，符合“RI 不对外承诺”。

1.10. 本次操作与当前状态

1. **新建** `note/2026-10-26.typ`（answer-note 完整头部），写入本主题。
2. **讲解内容**：值层、结构层、生命周期层、边界层、规格层、工程层六个层面的核心问题、典型子句、Go 检查方式与失败后果；汇总表；对 `mem` 只列六行问题，不填答案。
3. **未做**：没有改 `internal/db/kv.go` 与 `kv_test.go`；没有给 `KV` 的 RI 内容。
4. **验证**：`typst compile` 零错误；`go vet ./...` 通过。
5. **下一步**：按六个层面逐层过一遍你的 `mem`，把每层导出的子句写进注释与 `checkRep`；写完交作业，我按契约诊断。

1.11. 单键谓词与键集合的全称量化

这个观察很关键：**“单个 key 的合法性”与“所有 key 都合法”不是同一层的东西**。同时想两者会累，是因为它们在脑子里占两个工作记忆槽；工程上的解法不是“更努力地同时想”，而是**分层**：把单键规则写成一个谓词，把“所有”写成量化，把检查写成遍历。

1.11.1. 一、两种视角其实是“谓词”与“量词”

设 `P(k)` 表示“键 `k` 合法”。那么：

1. 单键视角：`P(k)` 对某一个具体的 `k` 成立吗？
2. 集合视角：`forall k in dom(m), P(k)` 对整个键集合成立吗？

后者的真值完全由前者决定；它们是同一件事的两个抽象层级，不是两条独立的规则。写 RI 时只需要：

> 📌 **核心要点 规则写一次（谓词 `P`），量化写一次（`forall k in dom(m), P(k)`），检查写一次（遍历 + 调用 `P`）。**三处共享同一个 `P`，心智负担就从“同时想两件事”变成“每次只想一层”。

1.11.2. 二、三层分离

层	关注点	产出
谓词层	单个 key 什么算合法？	一个 `validKey(k) bool` 函数

量化层	所有已存在的 key 是否都满足？	RI 注释里的一条全称量化
检查层	怎么在代码里验证？	`checkRep` 里的 `for k := range m` + 调用 `validKey`

入口校验是**第四个使用点**，但它复用同一个谓词，不重新定义规则。

1.11.3. 三、带背景的例子：用户索引表

场景：一个内部索引 `map[string]int`，键是用户名，值是该用户的 ID。用户名来自外部输入，规则是：非空、只含字母/数字/下划线、长度不超过 32 字节。

约束从哪来：非法用户名会污染索引；空用户名常与“未设置”的哨兵值混淆；长度上限防止存储膨胀。规则必须在“写入时”和“已存在的数据上”同时成立。

不遵守会怎样：如果只在 `Set` 里校验，而 `Get`、`Del`、`checkRep` 各自不校验，那么将来从别的路径写入（反序列化、测试直接改 map、迁移脚本）就会把非法键带进 rep，而且没有任何地方能发现。

与项目的关系：`KV` 的 key 是 `[]byte` 转成的 `string`；“空键是否合法”就是单键谓词；RI 是对 `dom(mem)` 的全称量化；`Set` 的入口校验与 `checkRep` 可以共享同一个谓词。

```
1    // validKey 是「单个 key」的谓词：只定义一次。
2    func validKey(k string) bool {
3      if k == "" || len(k) > 32 {
4        return false
5      }
6      for _, r := range k {
7        if r == '_' || r >= 'a' && r <= 'z' || r >= 'A' && r <= 'Z' || r >=
'0' && r <= '9' {
8          continue
9        }
10       return false
11     }
12     return true
13   }
```

Listing 4: 谓词层：单键规则只在这里出现一次；`len` 是字节长度，`range` 遍历的是 rune，两者语义不同但本例都只用于规则判断

```
1    // Rep invariant:
2    //   m is non-nil
3    //   for all k in dom(m), validKey(k)
```

Listing 5: 量化层：RI 只写“每个键都满足谓词”，不重复谓词内部的细节

```go
func (ix *Index) checkRep() {
  if ix.m == nil {
    panic("demo: nil index")
  }
  for k := range ix.m {
    if !validKey(k) {
      panic("demo: invalid key " + k)
    }
  }
}
```

Listing 6: 检查层：遍历键集合 + 调用同一个谓词；一条 RI 子句对应一个循环

```go
func (ix *Index) Set(name string, id int) error {
  if !validKey(name) {
    return fmt.Errorf("invalid username %q", name)
  }
  ix.m[name] = id
  ix.checkRep()
  return nil
}
```

Listing 7: 入口校验复用同一个谓词：返回 `error` 表示“可预期的输入错误”，与 `checkRep` 的 `panic`（程序 bug）分工不同

1.11.4. 四、前置条件与 RI：同一个谓词，两个使用点

同一个 `validKey` 可以同时服务两个目的：

1. **方法入口**：检查调用方传进来的参数，失败返回 `error`——这是**前置条件（precondition）**的运行时落实；
2. `checkRep`：检查 rep 里已存在的键，失败 `panic`——这是**表示不变式（RI）**的落实。

它们共享规则，但语义不同：

使用点	检查对象	失败时	属于
方法入口	调用方传入的参数	返回 `error`（可恢复）	前置条件 / 输入校验
`checkRep`	rep 里已存在的数据	`panic`（程序 bug）	RI
测试	构造出的 rep 值	`t.Fatalf`	验证 RI 与构造逻辑

> ⚠️ **陷阱预警** 不要因为“两者用同一个谓词”就把它们混成一条规则。**RI 说的是“已存在的数据必须合法”，前置条件说的是“调用方必须传合法的东西”**。一个方法是拒绝非法输入，另一个是发现内部已经坏了。

1.11.5. 五、写 RI 时的分类法

把 RI 子句先分成两类，再写：

类型	形式	例子

全局子句	对整个 rep 的断言	`m != nil` ；键的数量不超过上限
逐元素子句	`forall x in 容器, P(x)`	所有键非空；所有值非负
跨字段子句	字段之间的关系	`len(keys) == len(vals)` ； `min <= max`

分类之后再写，就不会在“某个键”与“所有键”之间来回切换 **：逐元素子句只写量词和谓词名，谓词内部的细节留在谓词函数里。**

1.11.6. 六、对 `mem` 的映射（只列问题）

1. 单键谓词层： `mem` 的 key 有哪些规则？(空键、长度、字符集、前缀……)
2. 量化层：这些规则是否要对 `dom(mem)` 里的每个键都成立？
3. 入口层： `Set` / `Get` / `Del` 收到不满足谓词的键时，返回 `error` 还是当作未命中？
4. 检查层： `checkRep` 是否遍历 `mem` 并调用同一个谓词？

每一层只回答一个问题；不要在一层里同时处理其他层。

> 📌 **核心要点** 心智负担的来源不是“key 有两种身份”，而是“规则、量化、检查三件事挤在同一段文字里”。把它们拆成三层之后，单键与集合就不再需要同时想——写谓词时只想单键，写 RI 时只想量词，写检查时只想遍历。

1.11.7. 本次操作

1. 追加本小节：单键谓词 / 全称量化 / 检查三层分离、带背景的用户索引表示例、前置条件与 RI 的分工、RI 子句分类法、对 `mem` 的四层问题。
2. 未改 `internal/db/kv.go` 与 `kv_test.go` ；未给 `KV` 的 key 规则。
3. 验证： `typst compile` 零错误； `go vet ./...` 通过。

1.11.8. 谓词、量化与三层含义

一、谓词（predicate）是什么

谓词就是“带参数、返回真或假”的判断。它本身不是一个固定的真/假值；把参数代进去，才变成一句可以判真假的话。

1. 数学写法： `P(x)` ，例如 `P(n) = n > 0` ；
2. Go 写法： `func(n int) bool` ，例如 `func isPositive(n int) bool { return n > 0 }` 。

对比一下：

东西	数学	Go	能不能直接判真假
命题	`1 > 0`	`1 > 0`	能，已经是真/假
谓词	`P(n) = n > 0`	`func(n int) bool`	不能，要代入参数
代入后的命题	`P(3)`	`isPositive(3)`	能，结果是 true

所以“key 非空”如果写成 `len(k) > 0` ，它其实是一个**谓词**（含变量 `k` ），不是一条已经成立的命题。

二、量化（quantification）是什么

量化是把一个谓词应用到某个范围上，回答“这个范围里的元素整体上满足吗”。两种最基本：

1. **全称量化（universal quantification）**：`∀ x ∈ S, P(x)` ——集合 `S` 里的**每一个**元素都满足 `P`；
2. **存在量化（existential quantification）**：`∃ x ∈ S, P(x)` ——集合 `S` 里**至少有一个**元素满足 `P`。

Go 没有内置的 `∀` / `∃`，所以量化用循环实现：

```go
func allKeys(m map[string]int, p func(string) bool) bool {
  for k := range m {
    if !p(k) {
      return false
    }
  }
  return true
}

func anyKey(m map[string]int, p func(string) bool) bool {
  for k := range m {
    if p(k) {
      return true
    }
  }
  return false
}
```

Listing 8: `allKeys` 是全称量化、`anyKey` 是存在量化；谓词 `p` 作为参数传入，量化逻辑与规则本身分离

> 📌 **核心要点 谓词层回答“一个元素合不合格”，量化层回答“这个判断覆盖哪些元素”。** 同一套规则（谓词）可以配全称量化（每个都满足）、存在量化（至少一个满足），也可以配计数、过滤等其他用法；规则只写一次，量化方式按需要选择。

三、三层的关系

层	问题	数学	Go
谓词层	单个 key 什么算合法？	`P(k)`	`func validKey(k string) bool`
量化层	所有 key 是否都满足？	`∀ k ∈ dom(m), P(k)`	`allKeys(m, validKey)` 或 `for k := range m`
检查层	怎么在运行时验证？	同上，代入具体 rep	在 `checkRep` 里遍历并调用谓词

一句话：**谓词是“规则”，量化是“规则的适用范围”，检查是“把规则跑一遍”。**

四、带背景的例子：用户索引表

场景：内部索引 `map[string]int`，键是用户名，值是该用户 ID。用户名规则：非空、只含字母/数字/下划线、长度不超过 32 字节。

约束从哪来：非法用户名会污染索引；空用户名常与“未设置”混淆；长度上限防止存储膨胀。规则既要在写入时检查，也要对已存在的数据成立。

不遵守会怎样：把规则直接写进 RI 注释（“每个 key 非空且长度不超过 32 且只含…”），规则一旦调整就要改注释、`Set`、`checkRep` 三处；漏一处就出现“注释说合法、代码放行”的不一致。

与项目的关系：`KV` 的 key 规则属于谓词层；RI 是它的全称量化；`checkRep` 是检查层。三层分开后，规则只在一处定义。

```
// 谓词层：规则只写一次
func validKey(k string) bool {
  if k == "" || len(k) > 32 {
    return false
  }
  for _, r := range k {
    if r == '_' || r >= 'a' && r <= 'z' || r >= 'A' && r <= 'Z' || r >= '0' && r <= '9' {
      continue
    }
    return false
  }
  return true
}
```

Listing 9: 谓词层：只回答“单个 key 合不合格”，不涉及集合

```
// 量化层：RI 只写「每个 key 都满足谓词」
// Rep invariant:
//   forall k in dom(m), validKey(k)
```

Listing 10: 量化层：出现 `forall`，但不重复谓词内部的细节

五、空集合上的全称量化

一个容易困惑但很重要的事实：**空集合上的全称量化恒为真**。

1. 数学上：`∀ k ∈ ∅, P(k)` 为真——因为没有反例；
2. Go 里：`for k := range m` 在空 map 上一次都不执行，`allKeys` 直接返回 `true`。

这正好符合 RI 的直觉：空 map 里没有非法键，所以“所有键都合法”成立。反过来，**存在量化在空集合上恒为假**——空 map 里找不到任何满足条件的键。

> ⚠️ **陷阱预警** 不要把“空集合上全称量化恒真”误当成 bug。“所有键非空”在空 map 上成立，不意味着“空 map 等于非空键”；它只是说没有反例。这条性质也让 RI 在对象刚创建、还没有数据时自然成立。

六、常见混淆

1. **把量化当谓词**：“所有 key 非空”是一个量化后的命题，不是谓词；谓词必须带参数；
2. **把谓词当量化**：`len(k) > 0` 只描述一个 `k`，没有“所有”；
3. **量词用错**：RI 需要全称量化（每个都满足），不是存在量化（有一个满足）；
4. **忘记空集合**：`allKeys` 在空集合上返回 `true`，这是正确行为；
5. **在 Go 里手写循环时漏掉 `return true`**：空集合会意外返回 `false`。

七、对 `mem` 的映射

1. 谓词层：`validKey(string) bool` —— `mem` 的 key 规则；
2. 量化层：`∀ k ∈ dom(mem), validKey(k)` ——写进 RI；
3. 检查层：`for k := range mem { if !validKey(k) { ... } }` ——写进 `checkRep`；
4. 入口层：`Set` / `Get` / `Del` 用同一个谓词做参数校验，失败返回 `error`。

> 📌 **核心要点** 谓词、量化、检查是三种不同的东西：谓词是“一条规则”，量化是“规则覆盖的范围”，检查是“在代码里把范围跑一遍”。把它们拆开后，写 RI 时只需要在“量化层”写一句 `forall ... , 谓词名`，不必同时回忆规则细节。

2. 讲解偏好：例子要带背景

这条反馈已记进项目指令（`AGENTS.md` 的「讲解偏好」）。以后举例默认给四个背景要素，再给代码：

要素	要回答什么
场景	这个类型/函数在什么系统里出现？解决什么问题？
约束从哪来	为什么需要这条规则？它防的是什么？
不遵守会怎样	一个具体的反例或故障，不是“可能出错”
与项目的关系	相同点与不同点；映射回 `internal/db` 时边界在哪

2.1. 示范一：`RatNum` 的 RI（结构层）

场景：任意精度有理数类，常见于金融计算、比例运算、需要避开浮点误差的场合。它用两个整数（分子、分母）表示一个精确分数。

约束从哪来：分母为 0 没有数学意义；不约分则同一个有理数有多种表示（`1/2` 与 `2/4`），会让相等判断、哈希、排序、作为 map 键都变得麻烦。课程因此把“分母为正 + 已约分”写进 RI。

不遵守会怎样：如果只检查 `den != 0` 而不要求约分，`RatNum{1,2}` 与 `RatNum{2,4}` 的抽象值相同，但字段不同；把它们当 map 的键会得到两个键，`==` 也会判不等。这不是“数值算错”，而是“同一个值有多个身份”。

与项目的关系：`KV` 的 map 也有“同一个抽象键值表、多种 rep”的现象（容量、哈希种子、插入顺序不同），但 `KV` 不需要“约分”这种规范化；要分清哪些 RI 子句是“合法性”（没有它就无抽象值），哪些是“唯一性/规范化”（没有它抽象值仍成立，但实现更难推理）。

2.2. 示范二：map 的空 key（值层）

场景：配置表或索引表，键是标识符（主机名、环境变量名、字段名）。这类表通常约定“键不能为空”。

约束从哪来：空键在业务上没有意义；更麻烦的是 `""` 经常被用作“未设置”的哨兵值，如果允许空键，就无法区分“显式设置了一个空键”与“这个键从未出现”。

不遵守会怎样：`Set("", v)` 之后 `Get("")` 命中；调用方拿到 `v`，却无法判断它是“空键的值”还是“默认值”。如果上层再用 `""` 表示“未配置”，两者就撞在一起。

与项目的关系：`KV` 的键是 `[]byte` 转成 `string`；`[]byte(nil)` 与 `[]byte{}` 都会变成 `""`，所以“空键”是一个不可再分的 rep 状态。是否禁止它，是规格决策；禁止之后，`Set` / `Get` / `Del` 对空键的行为都要跟着定义。

> 📌 **核心要点** 背景不是装饰。它决定这条 RI 是“从业务约束推出来的”，还是“看起来合理但会误伤合法输入”。没有背景的规则，很容易写成过强的 RI。

2.3. 本次操作

1. 在 `AGENTS.md` 新增「讲解偏好：例子要带背景」一节，列出四个要素。
2. 在 `note/2026-10-26.typ` 追加本小节与两个带背景的示范（`RatNum`、map 空 key）。
3. 未改 `internal/db/kv.go` 与 `kv_test.go`；未给练习答案。
4. 验证：`typst compile` 零错误；`go vet ./...` 通过。

3. 测试分区的顺序：先 subdomain，后 cartesian product

3.1. 费曼直觉

subdomain 是“一个维度上切出来的一块”；**cartesian product** 是“把几个维度切出来的块两两相乘”。顺序上先有“每块怎么切”，才谈得上“块与块怎么组合”——就像先给每个参数的取值分类，再讨论“参数 A 的哪类 × 参数 B 的哪类”。

如果把两者同时想，工作记忆会同时装“每个维度的规则”和“维度之间的组合”，很容易漏维度或重复。分层之后，每一步只做一件事。

3.2. 硬核机制

3.2.1. 一、三个概念

概念	定义	作用范围
subdomain（子域）	一个维度上的一类输入，满足不交、完备、非空	单维度
partition（分区）	一个维度上所有子域的集合	单维度
cartesian product（笛卡尔积）	多个维度的子域组合成的元组集合	多维度

关键点：**分区是单维度的划分，笛卡尔积是分区之间的组合**。没有单维分区，“积”没有定义。

3.2.2. 二、推荐顺序

1. **列出维度**：哪些参数、状态、操作序列会影响行为？
2. **对每个维度单独分区**：切出 subdomain，检查不交、完备、非空；只包含合法输入。
3. **单维覆盖**：每个子域至少选一个代表用例，形成基线。
4. **做笛卡尔积**：把各维度的分区相乘，得到组合子域。
5. **裁剪组合**：按规格依赖、风险、成本决定哪些组合必须测、哪些可以合并或跳过。
6. **补序列维度**：操作顺序、初始化状态、重复操作通常单独作为一个维度。
7. **写测试表**：每个用例标注它覆盖的子域/组合；复查覆盖与重复。

> 📌 **核心要点 先分区，后组合，再裁剪。**分区保证“每个维度不漏”，笛卡尔积保证“维度交互被考虑”，裁剪保证“成本可控”。顺序反过来（先列组合再想分区）会导致两个问题：维度没定义全，组合也没有边界。

3.2.3. 三、为什么先 subdomain

1. **定义顺序**：笛卡尔积作用于集合；子域是集合的元素。没有子域，积无从谈起。
2. **覆盖保证**：单维分区的不交/完备/非空，是后面所有组合的地基；地基漏了，组合再多也补不回来。

3. **认知负担**：先单维时只关心一个维度；做积时只关心“哪些维度需要交叉”；裁剪时只关心“哪些组合高风险”。每步只回答一个问题。

3.2.4. 四、什么时候必须做笛卡尔积

不是所有维度都需要全组合。判断依据是**规格是否对组合定义了不同行为**：

1. 维度之间**独立**：规格对每个维度单独定义行为，交互不影响结果 → 单维覆盖 + 少量交叉即可；
2. 维度之间**强依赖**：行为由多个维度共同决定（例如“账户状态 × 金额上限”）→ 必须测交叉；
3. 组合数**可控**：维度少（≤3）且子域不多 → 全组合；
4. 组合数**爆炸**：维度多 → 用裁剪策略。

3.2.5. 五、裁剪策略

策略	做法	保证什么	代价
全组合	所有子域相乘	所有交互都覆盖	指数级用例数
单维基线 + 交叉抽样	每个子域至少一次，再补高风险交叉	单维完备 + 关键交互	可能漏掉低风险交叉
pairwise	保证任意两个维度的组合至少出现一次	两两交互覆盖	不保证三阶及以上交互
等价类合并	行为相同的子域合并	减少重复	需要确认行为真的相同

> ⚠️ **陷阱预警** 裁剪不是“随便删用例”。每删一个组合，都要能回答：“这个组合的行为与保留的哪个组合等价？”答不上来就只能保留，或者把它标成已知缺口。

3.2.6. 六、带背景的例子：转账函数

场景：一个 `Transfer(from, to Account, amount Money) error` 函数，用于账户间转账。输入维度：账户状态（正常/冻结/不存在）、金额（0/正/负/超限）、币种（同币种/跨币种）。

约束从哪来：账户状态决定能否扣款；金额决定是否合法；币种决定是否需要汇率转换。三者的组合决定最终行为，单看任何一个都不够。

不遵守会怎样：如果只测单维（正常账户转正数、冻结账户转正数……），会漏掉“冻结账户 + 跨币种”这种组合 bug；如果直接测 $3\times4\times2=24$ 种组合，成本高且大量重复（例如“账户不存在”时金额与币种都无关）。

与项目的关系：`KV` 的维度类似——key 的状态、value 的状态、操作序列、是否已 `Open`。先给每个维度分区，再决定哪些交叉必须测；不要一上来就列“所有组合”。

```go
func TestTransfer(t *testing.T) {
  tests := []struct {
    name   string
    status string
    amount int
    want   error
  }{
    {"active positive", "active", 100, nil},
    {"frozen positive", "frozen", 100, ErrFrozen},
    {"missing positive", "missing", 100, ErrNoAccount},
    {"active zero", "active", 0, ErrZeroAmount},
  }

  for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
      if err := Transfer(tt.status, tt.amount); err != tt.want {
        t.Fatalf("Transfer(%q, %d) = %v, want %v", tt.status, tt.amount,
err, tt.want)
      }
    })
  }
}
```

Listing 11: 表驱动测试：用例名写清维度组合，失败消息包含输入与期望；`go doc testing.T.Run` 说明子测试在独立 goroutine 中运行

3.2.7. 七、与 RI 的类比

测试侧	RI 侧	共同点
subdomain	单键谓词 `P(k)`	先定义单元素规则
cartesian product	全称量化 `forall k, P(k)`	再把规则扩展到集合/组合
裁剪	只写有意义的 RI 子句	去掉不产生额外约束的部分
代表用例	一次 `checkRep` 调用	用少量检查覆盖规则

两者都是“先定义局部规则，再组合到全局”；顺序反了，就会在全局层面反复猜局部规则。

3.2.8. 八、对当前项目的问题清单（不填答案）

1. 维度有哪些？(key 的状态、value 的状态、操作序列、初始化状态、是否关闭)
2. 每个维度的合法子域怎么切？是否不交、完备、非空？
3. 哪些维度之间是强依赖？哪些组合行为等价、可以合并？
4. 组合总数是多少？是否需要 pairwise 或风险裁剪？
5. 哪些交叉是高风险（删除后查询、覆盖写、空值、未初始化）？
6. 每个保留的组合，测试名与失败消息是否能说清“覆盖了哪个子域”？

> 📌 **核心要点** 顺序记成一句话：**先切每个维度（subdomain），再乘出组合（cartesian product），最后按规格与风险裁剪。** 分区是设计，笛卡尔积是组合，裁剪是成本控制；三者不能合并成一步。

3.3. Go 语言机制补充

1. 表驱动测试用 `t.Run` 给每个组合命名；`go test -run TestXxx` 可按名字过滤（`go doc testing.T.Run`）。
2. 失败消息包含“函数名 + 输入 + got + want”，否则组合一多就无法定位。
3. 子测试可以 `t.Parallel()`，但共享状态时要小心；`KV` 的测试目前是串行的。
4. `go test -cover` 只能告诉你“代码执行过”，不能告诉你“子域覆盖了”；覆盖率与分区覆盖是两回事。
5. `go test -run` 配合 `-v` 可以只跑某一类组合，便于调试。

3.4. 本次操作与当前状态

1. 新增本主题：subdomain / partition / cartesian product 的定义、推荐顺序（分区 → 组合 → 裁剪）、裁剪策略、转账函数的带背景示例、与 RI 的类比、对当前项目的问题清单。
2. 未改 `internal/db/kv.go` 与 `kv_test.go`；未给 `KV` 的具体分区答案。
3. 验证：`typst compile` 零错误；`go vet ./...` 通过。

3.5. 边界属于哪个阶段：分区，而不是列维度

结论先说：**“Include boundaries in the partition” 属于分区阶段**，而且是分区的一个**细化步骤**；它不属于“列出维度”。

3.5.1. 一、为什么不是“列维度”

维度是输入空间的坐标轴：key 的状态、value 的状态、操作序列、是否初始化。**边界**是某条坐标轴上的一个点或端点：空字符串、nil、0、上限、序列首尾。把边界当成维度，会让坐标轴数量爆炸，而且每条轴上的其他值仍然没有分类。

阶段	做什么	边界的角色	产物
列出维度	确定坐标轴：哪些参数/状态影响行为	不涉及	维度清单
分区	每条轴切子域；把边界拆成单元素子域	边界是轴上的端点，单列成子域	每个维度的分区
选代表	每个子域挑一个用例	边界子域只有一个元素，必然被选中	基线用例
组合	维度相乘	边界组合通常是高风险	组合子域
裁剪	合并/删除组合	边界组合优先保留	测试表

3.5.2. 二、材料里的做法

6.102 R02 的 “Include boundaries in the partition” 是 “Choosing test cases by partitioning” 的子节。它的做法不是“另加一个边界维度”，而是：

1. 先把每个维度按行为切成子域；
2. 再把边界值从相邻子域中**拆出来**，变成**单元素子域（single-element subdomain）**；

3. 这样“每个子域选一个代表”时，边界值一定会被选中。

代价是：拆出边界后，相邻子域要相应收缩，否则会重叠（违反不交）。

3.5.3. 三、操作顺序

1. 按行为切子域（不交、完备、非空）；
2. 对**每个维度**找边界：空集合、0、类型极值、序列首尾、长度上限；
3. 把每个边界拆成单元素子域，同时从原相邻子域中移除该点；
4. 复查三性质：不交（边界不在两个子域里）、完备（边界没被漏掉）、非空（每个子域至少一个合法值）；
5. 进入选代表与组合阶段。

> 📌 **核心要点** 边界是**分区阶段的修正**，不是独立的阶段，也不是维度。它回答的是“这条轴上的端点有没有被单独覆盖”，而不是“还有没有别的轴”。

3.5.4. 四、带背景的例子：分页函数

场景：`Page(items []Item, offset, limit int) ([]Item, error)`，用于分页读取列表。约束：`offset >= 0`；`limit` 在 $[1, \text{MaxLimit}]$ 之间；`offset` 超过列表长度时返回空页。

约束从哪来：分页最容易出 off-by-one；`offset = 0`、`limit = 1`、`limit = MaxLimit`、`offset = len(items)` 都是行为变化的点。规格只对这些点附近的边界敏感。

不遵守会怎样：如果只按“offset 小/大、limit 小/大”切子域，再随机选代表（例如 offset=5、limit=10），可能永远测不到 `offset=0` 与 `limit=1`，而这两个正是最容易写错的下标。

与项目的关系：`KV` 的边界同样是“某条轴上的点”——空 key、nil 值、空 map、删除后再查、未 `Open`。它们应该在分区阶段被拆成单元素子域，而不是被当成新的维度。

```
// 维度：offset、limit
// offset 的边界：0、len(items)、len(items)+1
// limit 的边界：1、MaxLimit、MaxLimit+1
// 每个边界单独成子域；其余按行为切成 "中间值" 子域
```

Listing 12: 边界是每个维度上的端点，拆成单元素子域；不是新增维度

3.5.5. 五、对当前项目的边界问题清单（不填答案）

1. 每条维度（key、value、操作序列、初始化状态）各自的边界是什么？
2. 空 key / nil 值 / 空 map 是边界，还是“另一种维度”？
3. 删除后再查、重复删除、覆盖写，属于序列维度的边界吗？
4. 拆出边界后，相邻子域是否同步收缩？三性质还成立吗？
5. 边界组合（例如“空 key × 删除后查询”）是否属于高风险，必须保留？

3.5.6. 六、常见错误

1. **把边界当维度**：“空 key 维度”其实是 key 维度上的一个单元素子域；
2. **只在子域里包含边界**：不单列就不保证被选中；
3. **拆出边界后忘记收缩相邻子域**：边界同时属于两个子域，违反不交；

4. **只找数值边界**：忘了空集合、空字符串、序列首尾、nil；
5. **边界组合全保留**：边界本身已经很多，组合会爆炸；应按风险裁剪；
6. **把非法输入当边界**：前置条件排除的输入不属于分区对象。

> ⚠️ **陷阱预警** “边界”与“非法输入”是两回事。边界仍然在合法输入空间内（空字符串如果被规格允许，就是合法边界）；非法输入属于前置条件的领地，不应混进分区。

3.5.7. 七、与 RI 的类比

1. 分区像 RI 的逐元素谓词：先定义单条轴上的规则；
2. 边界像 RI 里的特殊 rep 值：nil map、空集合、零值——它们往往需要单独子句；
3. 拆边界像“把恒真/通用子句拆成具体检查”：让规则可执行、可覆盖。

3.6. 本次操作与当前状态

1. 追加本小节：边界属于分区阶段而非列维度、材料做法（单元素子域）、操作顺序、分页函数的带背景示例、对当前项目的边界问题清单、常见错误。
2. 未改 `internal/db/kv.go` 与 `kv_test.go`；未给 `KV` 的具体边界划分。
3. 验证：`typst compile` 零错误；`go vet ./...` 通过。

3.7. Using multiple partitions 与 cartesian product 的关系

结论：**两者不互斥，但不是同一层的东西。** `Using multiple partitions` 是“多个分区并存、用少量用例覆盖各子域”的设计策略；`cartesian product` 是“把多个分区相乘”的组合策略。材料原文明确把前者称为后者的**替代方案**（alternative approach），但也说可以在多个分区上**有选择地**使用笛卡尔积。

3.7.1. 一、材料怎么说

6.102 R02 的 “Using multiple partitions” 一节有三个关键句：

1. 对多参数函数，把所有参数的子域做笛卡尔积会导致**组合爆炸**（n 个参数时用例数指数增长）；
2. 替代做法是**对每个参数分别分区**，一个测试用例可以同时覆盖多个分区的子域——材料用 6 个用例覆盖了两个各有 6 个子域的分区，而不是 36 个；
3. 但独立分区会**漏掉参数之间的交互**，所以可以再加一个“交互分区”（例如两个数的符号组合），仍然不需要全积。

材料同时留了一个口子：

> Sometimes we may want to use the Cartesian product approach on multiple partitions, to produce a more thorough test suite. But even in those cases, the Cartesian product may be smaller than we expect. When subdomains from different partitions turn out to be mutually exclusive, the Cartesian product won’t include a subdomain for that particular combination of subdomains.

翻译：有时确实想对多个分区做笛卡尔积，得到更彻底的套件；但即便那时，积也可能比预期小——互斥的子域组合不会出现在积里。

3.7.2. 二、两者的分工

策略	回答的问题	优点	风险
Using multiple partitions	用哪些分区？每个分区怎么切？	用例少；一个用例可覆盖多个子域	可能漏交互，需要额外交互分区
Cartesian product	把哪些分区的子域相乘？	交互覆盖彻底	组合爆炸；很多组合可能无意义

它们的关系可以记成：

> 📌 **核心要点 multiple partitions 是“分区设计”，cartesian product 是“组合求解”。**前者决定“有哪些维度、每个维度怎么切”，后者决定“这些维度的子域要不要相乘、相乘后保留哪些”。前者可以独立使用，也可以作为后者的输入；两者不是互斥的选项。

3.7.3. 三、它们各自属于哪个阶段

按上一节的阶段划分：

1. **列出维度**：确定坐标轴；
2. **分区**：每条轴切子域；`Using multiple partitions` 属于这一阶段的**设计思路**（允许多个分区并存，而不是硬凑成一个全积分区）；
3. **组合**：`cartesian product` 属于这一阶段（把分区相乘）；
4. **裁剪**：决定保留哪些组合。

所以严格说，**multiple partitions 不属于组合层面**，它属于分区设计；只是它会影响组合策略的选择。宽泛地说，两者都在“处理多个维度”这个大层面，但阶段不同。

3.7.4. 四、带背景的例子：乘法

场景：`multiply(a, b)` 计算两个整数的乘积，输入是两个数。

约束从哪来：符号处理、`0` / `1` 的边界、大数溢出是 bug 高发点；两个参数各有自己的行为变化。

不遵守会怎样：如果只做一个“a × b 的笛卡尔积分区”，每个参数 6 个子域，就要 $6 \times 6 = 36$ 个用例；如果只做 multiple partitions 而不加交互分区，又可能漏掉“正 × 负”的符号 bug。

与项目的关系：`KV` 的 key、value、操作序列可以分别分区，用少量用例覆盖各子域；对高风险交互（删除后查询、覆盖写、空值）额外加“交互分区”或对子集做笛卡尔积，而不是全积。

```
// partition on key:
//   key exists
//   key missing
//   key is empty
//   key is nil
//
// partition on operation sequence:
//   Set then Get
//   Set then Del then Get
//   Del twice
//
// 这两个分区可以独立覆盖；交互风险（例如 delete-then-get）
// 再单独作为一个分区或交叉组合来处理。
```

Listing 13: multiple partitions 的写法：每个维度一个分区，允许一个用例同时覆盖多个分区的子域；交互风险另加分区

3.7.5. 五、操作建议

1. 先给每个维度分区；
2. 用 multiple partitions 做基线：每个子域至少被一个用例覆盖，允许一个用例覆盖多个子域；
3. 找出交互风险，增加“交互分区”（例如“两个参数的符号组合”）；
4. 只对高风险维度子集使用笛卡尔积，而不是全积；
5. 边界值按上一节的方法单列成子域；
6. 记录被裁剪掉的组合，作为已知缺口。

3.7.6. 六、常见错误

1. **认为两者互斥**：材料明确说可以混合；
2. **认为 multiple partitions 就是“完全不交叉”**：独立分区会漏交互，需要交互分区补；
3. **认为必须全积**：材料用 6 个用例覆盖了 6+6+5 个子域；
4. **把一个用例覆盖多个子域当成重复**：这是 multiple partitions 的效率来源；
5. **忘记互斥组合**：笛卡尔积可能因为子域互斥而变小；
6. **把交互分区当成新维度**：它是对已有维度组合的补充划分，不是新的输入参数。

> ⚠️ **陷阱预警** multiple partitions 的“少用例”不是免费的：它用**交互覆盖**换**用例数量**。如果你的 bug 主要出现在维度交叉处，就必须补交互分区或做局部笛卡尔积；否则测试套件会“每个子域都覆盖，但组合全漏”。

3.8. 本次操作与当前状态

1. 追加本小节：multiple partitions 与 cartesian product 不互斥、材料原文要点、两者分工表、阶段归属、乘法带背景示例、操作建议、常见错误。
2. 未改 `internal/db/kv.go` 与 `kv_test.go`；未给 `KV` 的具体分区方案。
3. 验证：`typst compile` 零错误；`go vet ./...` 通过。

3.9. 操作分类（creator / producer / observer / mutator）在哪个阶段

先纠正一个术语：材料里的四分类是 **creators / producers / observers / mutators**（创建者 / 生产者 / 观察器 / 变值器），不是 "viewer"。观察器对应 `Get`、`Len` 这类只读操作。

结论：**操作分类不是"维度 → 分区 → 选代表 → 组合 → 裁剪"五步中的某一步，而是这五步的外层组织框架**。它回答"要测哪些操作"；五步回答"每个操作的输入空间怎么测"。其中"操作序列"可以作为输入空间的一个维度，进入五步。

3.9.1. 一、四类操作

类型	做什么	测试重点	与 RI 的关系
creator	创建新对象（`New`、`Open`）	初始状态合法；RI 建立	建立 RI
producer	从旧对象产生新对象（`Clone`、`Add` 返回新值）	新对象独立；不共享 rep；RI 成立	建立 RI（新对象）
observer	观察但不改变（`Get`、`Len`）	抽象值不变；只读	保持 RI
mutator	修改当前对象（`Set`、`Del`）	状态转换；失败路径；RI 保持	保持 RI

> 📌 **核心要点** 操作分类解决的是**测试对象的选择**：每个 creator、producer、observer、mutator 都至少要被测到。它不是输入空间的维度；输入空间是"某个操作的参数与调用历史"。

3.9.2. 二、放到五步框架里

阶段	操作分类的角色	说明
维度	操作序列可以是一个维度；操作类型本身不是	输入空间关心"调用了哪些操作、顺序如何"，不关心"这个操作属于哪一类"
分区	对每个操作的参数分别分区	creator 的参数、mutator 的参数各有自己的子域
选代表	每个操作 × 每个子域至少一个用例	可以用测试矩阵：行 = 操作，列 = 子域/边界
组合	跨操作的序列组合（`Set` → `Del` → `Get`）	序列维度参与组合；操作类型不参与笛卡尔积
裁剪	按操作风险裁剪	creator/producer 的 RI 检查、mutator 的失败路径优先保留

所以更准确的说法是：

```
for each operation in {creators, producers, observers, mutators}:
    list dimensions of its inputs
    partition each dimension (including boundaries)
    pick representatives
    combine sequences where interactions matter
    trim by risk
```

Listing 14: 操作分类是外层循环；五步是循环体；操作序列是循环体里的一个维度

3.9.3. 三、为什么不能把操作类型当维度

如果把 creator/producer/observer/mutator 当成一个维度塞进笛卡尔积，会出两个问题：

1. **语义不对**：操作类型不是输入值，它是“被测函数”的选择；把函数选择当输入维度，组合出来的“用例”没有意义；
2. **组合爆炸**：操作类型 × 每个操作的参数子域 × 序列，会产生大量重复或不可执行的组合。

正确的做法是：**外层按操作分类组织测试，内层对每个操作做五步**；需要跨操作交互时，把“操作序列”作为一个独立维度。

3.9.4. 四、带背景的例子：Counter ADT

场景：一个计数器 ADT，包含 `New() Counter`（creator）、`Add(n int)`（mutator）、`Value() int`（observer）、`Clone() Counter`（producer）。

约束从哪来：计数器需要保证“值只增不减”或“允许负数”；`Clone` 必须返回独立对象，否则两个计数器共享内部状态；`Add` 失败时不能破坏已有值。不同操作类型的 bug 形态不同，所以测试要分类组织。

不遵守会怎样：只测 `Value` 很容易，但永远发现不了 `Add` 破坏 RI 或 `Clone` 共享 rep；只测 creator 又覆盖不到状态转换。漏掉 producer 的“独立性”检查，两个对象会悄悄共享底层数组。

与项目的关系：`KV` 的 `Open` 是 creator，`Set` / `Del` 是 mutator，`Get` 是 observer，目前没有 producer。测试应按这四类组织；`Set` 后 `Get`、`Set` 后 `Del` 后 `Get` 是序列维度，不是操作类型维度。

```
// Testing strategy for Counter:
//   creator: New -> initial value is 0; checkRep holds
//   mutator: Add(n) -> partition n into {0, positive, negative};
//             checkRep after each call
//   observer: Value -> does not change the abstract value;
//             call before/after Add and compare
//   producer: Clone -> returned Counter is independent;
//             mutating the clone must not affect the original
//
// operation sequence (a dimension, not an operation type):
//   New -> Add -> Value
//   New -> Clone -> Add(clone) -> Value(original)
```

Listing 15: 按操作分类组织测试策略；序列单独作为维度，不混进操作类型

3.9.5. 五、与 RI 的对应

1. **creator / producer**：建立 RI。测试要在创建后调用 `checkRep`，并验证初始抽象值。
2. **mutator**：保持 RI。测试要在每次修改后调用 `checkRep`，并覆盖失败路径。
3. **observer**：保持 RI，且不改变抽象值。测试可以在调用前后比较抽象值（或调用 `checkRep`）。
4. **producer**：新对象的 RI 要成立，且与原对象不共享 rep（表示暴露检查）。

3.9.6. 六、常见错误

1. **把操作类型当输入维度**：组合爆炸且语义错误；
2. **只测 observer**：最容易写，但覆盖不到状态变化；
3. **忘记 producer**：`Clone` / `Add` 返回新对象时，共享 rep 的 bug 很难发现；
4. **操作序列与操作类型混为一谈**：序列是输入空间维度，类型是测试范围；
5. **每个操作重复写相同的分区**：把共享的分区抽出来复用，避免不一致；
6. **只测成功路径**：mutator 的失败路径（非法参数、前置条件不满足）也要覆盖。

> ⚠️ **陷阱预警** 操作分类不是“多写几个测试”的同义词。它的价值在于**覆盖不同的 bug 形态**：creator 的初始状态、mutator 的状态转换、observer 的只读性、producer 的独立性。缺哪一类，就缺一类 bug 的发现能力。

3.10. 本次操作与当前状态

1. 追加本小节：四类操作的定义与测试重点、五步框架中的位置（外层循环 + 序列维度）、不能把操作类型当维度的原因、Counter 的带背景示例、与 RI 的对应、常见错误。
2. 未改 `internal/db/kv.go` 与 `kv_test.go`；未给 `KV` 的具体测试方案。
3. 验证：`typst compile` 零错误；`go vet ./...` 通过。

3.11. Testing an abstract data type 与五步的区别和联系

结论：**两者不是替代关系，而是“外层框架”与“内层流程”的关系。** `Testing an abstract data type`（ADT 测试）以**操作为单位**组织测试，处理操作之间的依赖；五步（维度 → 分区 → 选代表 → 组合 → 裁剪）处理**单个操作的输入空间**。五步是 ADT 测试的循环体。

3.11.1. 一、材料怎么说

6.102 R06 的 “Testing an abstract data type” 开篇两句话点明了核心：

> We build a test suite for an abstract data type by creating tests for each of its operations. These tests inevitably interact with each other. The only way to test creators, producers, and mutators is by calling observers on the objects that result, and likewise, the only way to test observers is by creating objects for them to observe.

翻译：ADT 的测试套件是**为每个操作创建测试**；这些测试不可避免地互相作用——测试 creator、producer、mutator 的唯一方式是**对产生的对象调用 observer**；测试 observer 的唯一方式是**先创建对象**让它观察。

材料随后给出 MyString 的测试策略：每个操作各自分区，且多个操作可以共享同一分区（DRY 写法）。例如：

```
// testing strategy for each operation of MyString:
//
// constructor, length, charAt, substring:
//    partition on string length: 0, 1, >1
// length, charAt, substring:
//    partition on this: produced by constructor, produced by substring
// charAt:
//    partition on i=0, 0<i<len-1, i=len-1
// substring:
//    partition on start=0, 0<start<len, start=len
//    partition on end=0, 0<end<len, end=len
//    partition on end-start: 0, >0
```

Listing 16: ADT 测试策略：按操作列出分区；共享分区合并写；`this` 的来源本身是一个维度

注意其中一条分区：`partition on this: produced by constructor, produced by substring` ——**对象是由哪个操作产生的**，这是 ADT 测试特有的维度，不是普通参数维度。

3.11.2. 二、区别

维度	Testing an ADT	五步	关系
单位	操作（creator/producer/observer/mutator）	单个操作的输入空间	外层 vs 内层
目标	覆盖四类操作 + 操作间依赖	覆盖单个操作的所有子域	互补
产物	测试策略大纲（每个操作一段）	分区表、代表用例、组合与裁剪	大纲是五步的容器
特有内容	夹具、验证链、对象来源、equality 约束	维度/分区/边界/组合/裁剪	ADT 测试多出的部分

五步没有消失，只是从“整个 ADT”降级为“每个操作内部”：

```
for each operation in {creator, producer, observer, mutator}:
    build fixture (objects from creators/producers)
    list dimensions (parameters + this-source)
    partition each dimension (including boundaries)
    pick representatives
    combine sequences where interactions matter
    trim by risk
```

Listing 17: ADT 测试是外层循环；五步是循环体；夹具与 `this` 来源是 ADT 测试额外加的两件事

3.11.3. 三、联系：ADT 测试多出的四件事

1. **操作分类**：外层循环按四类操作组织，保证每类都有测试；
2. **测试夹具（fixture）**：observer 不能凭空测，必须先用 creator/producer 造一个对象；mutator 的测试也要先有初始对象；
3. **验证链**：mutator 的效果只能用 observer 观察；creator/producer 的效果用 observer + `checkRep` 验证；observer 的只读性通过调用前后比较；
4. **对象来源维度**：`this` 由哪个操作产生（constructor / producer / 经过 mutator）本身是一个输入维度，进入分区与组合。

> 📌 **核心要点** ADT 测试 = **外层（操作分类 + 夹具 + 验证链 + 来源维度）** + **内层（五步）**。五步解决“每个操作的输入空间怎么覆盖”，ADT 测试解决“操作之间怎么互相验证、对象从哪来”。

3.11.4. 四、带背景的例子：MyString

场景：一个不可变字符串 ADT，操作有 constructor、length、charAt、substring。材料特意没有定义 equality。

约束从哪来：没有 equality，就不能直接用 `assert` 比较两个对象；只能用已定义的操作（length、charAt）观察它们。另外，substring 产生的对象与 constructor 产生的对象在“来源”上不同，可能触发不同 bug。

不遵守会怎样：材料里的 Louis 试图直接比较 `new MyString('bye').substring(0, 3)` 与 `new MyString('bye')`，测试失败——因为对象相等没有定义。正确做法是用 length 与 charAt 逐个字符验证。另一个坑是只测 constructor，不测 substring 产生的对象作为 `this` 的情况。

与项目的关系：`KV` 的 `Get` 是 observer，`Set` / `Del` 是 mutator，`Open` 是 creator。测试 `Set` 的效果必须用 `Get` 观察；测试 `Get` 必须先构造一个对象；`this` 的来源可以是“刚 `Open` 的空 KV”、“经过 `Set` 的 KV”、“经过 `Del` 的 KV”——这是 ADT 测试特有的状态维度。

```
// testing strategy for KV (sketch, not a solution):
//
// Open (creator):
//    initial state; Get on empty KV
// Set (mutator):
//    partition on key: exists / missing / empty
//    partition on value: non-empty / empty / nil
//    observe result with Get; checkRep after
// Del (mutator):
//    partition on key: exists / missing
//    observe result with Get; checkRep after
// Get (observer):
//    partition on this: empty KV / after Set / after Del
//    partition on key: exists / missing / empty
//    verify no mutation (call twice, compare)
//
// operation sequence: Set -> Get; Set -> Del -> Get; Del -> Del
```

Listing 18: ADT 测试策略骨架：按操作分类；每个操作内部再分区；`this` 来源与序列单独列出

3.11.5. 五、常见错误

1. **只测 observer**：最容易写，但覆盖不到 mutator 的状态转换；
2. **用 mutator 验证 mutator**：`Set` 之后又 `Set`，没有用 `Get` 观察结果；
3. **忘记夹具**：observer 测试没有先构造对象，或 mutator 测试没有初始状态；
4. **忽略 `this` 来源**：只测 constructor 产生的对象，漏掉 producer/mutator 之后的状态；
5. **直接比较对象**：没有定义 equality 时，比较的是引用/未定义行为；
6. **把操作类型当输入维度**：会组合爆炸，且语义错误；
7. **不检查 RI**：creator/producer/mutator 之后没有调用 `checkRep`。

> ⚠️ **陷阱预警** ADT 测试的“单位”是操作，但“验证手段”是 observer。测试 creator/producer/mutator 时，真正断言的地方几乎总在 observer 上；如果只写 mutator 调用而不观察结果，测试等于没测。

3.12. 本次操作与当前状态

1. 追加本小节：ADT 测试与五步的区别（外层框架 vs 内层流程）、材料原文要点、MyString 测试策略、ADT 测试多出的四件事（操作分类/夹具/验证链/来源维度）、对 `KV` 的策略骨架、常见错误。
2. 未改 `internal/db/kv.go` 与 `kv_test.go`；未给 `KV` 的具体测试用例。
3. 验证：`typst compile` 零错误；`go vet ./...` 通过。

Index of Listings

Listing 1 值层的最小例子：单个字段的范围约束；`Counter` 是通用示例 4
Listing 2 结构层的例子：第二条约束两个字段之间的关系；`RatNum` 来自课程材料 . . 4
Listing 3 生命周期层的示意：同一个 rep 在不同阶段有不同的合法状态；字段名是占位，不是项目代码 5
Listing 4 谓词层：单键规则只在这里出现一次；`len` 是字节长度，`range` 遍历的是 rune，两者语义不同但本例都只用于规则判断 9
Listing 5 量化层：RI 只写“每个键都满足谓词”，不重复谓词内部的细节 9
Listing 6 检查层：遍历键集合 + 调用同一个谓词；一条 RI 子句对应一个循环 10
Listing 7 入口校验复用同一个谓词：返回 `error` 表示“可预期的输入错误”，与 `checkRep` 的 `panic`（程序 bug）分工不同 10
Listing 8 `allKeys` 是全称量化、`anyKey` 是存在量化；谓词 `p` 作为参数传入，量化逻辑与规则本身分离 12
Listing 9 谓词层：只回答“单个 key 合不合格”，不涉及集合 13
Listing 10 量化层：出现 `forall`，但不重复谓词内部的细节 13
Listing 11 表驱动测试：用例名写清维度组合，失败消息包含输入与期望；`go doc testing.T.Run` 说明子测试在独立 goroutine 中运行 19
Listing 12 边界是每个维度上的端点，拆成单元素子域；不是新增维度 21
Listing 13 multiple partitions 的写法：每个维度一个分区，允许一个用例同时覆盖多个分区的子域；交互风险另加分区 24
Listing 14 操作分类是外层循环；五步是循环体；操作序列是循环体里的一个维度 . . 26
Listing 15 按操作分类组织测试策略；序列单独作为维度，不混进操作类型 26
Listing 16 ADT 测试策略：按操作列出分区；共享分区合并写；`this` 的来源本身是一个维度 28
Listing 17 ADT 测试是外层循环；五步是循环体；夹具与 `this` 来源是 ADT 测试额外加的两件事 28
Listing 18 ADT 测试策略骨架：按操作分类；每个操作内部再分区；`this` 来源与序列单独列出 30